

L20-L21: Information Flow Analysis

Software Analysis

IIT Guwahati · September 23-24, 2026

Does this program leak the PIN?

```
unsigned secret = read_pin();
unsigned pub = 0;
for (int b = 0; b < 32; b++)
    if (secret & (1u << b))
        pub = pub | 1u << b;
printf("%u\n", pub);
```

Does this program leak the PIN?

```
unsigned secret = read_pin();  
unsigned pub = 0;  
for (int b = 0; b < 32; b++)  
    if (secret & (1u << b))  
        pub = pub | 1u << b;  
printf("%u\n", pub);
```

No assignment ever copies
secret into pub.

Does this program leak the PIN?

```
unsigned secret = read_pin();  
unsigned pub = 0;  
for (int b = 0; b < 32; b++)  
    if (secret & (1u << b))  
        pub = pub | 1u << b;  
printf("%u\n", pub);
```

No assignment ever copies
secret into pub.

Yet it prints **the PIN**: pub ==
secret.

Does this program leak the PIN?

```
unsigned secret = read_pin();  
unsigned pub = 0;  
for (int b = 0; b < 32; b++)  
    if (secret & (1u << b))  
        pub = pub | 1u << b;  
printf("%u\n", pub);
```

Every bit travels through a **branch**.

No assignment ever copies secret into pub.

Yet it prints **the PIN**: `pub == secret`.

Secure Computing Systems

What Are We Protecting?

Overall goal: **secure the data** a system manipulates, by enforcing a **security policy**.

Secure Computing Systems

What Are We Protecting?

Overall goal: **secure the data** a system manipulates, by enforcing a **security policy**.

- **Confidentiality**: secret data does not leak to non-secret places.

Secure Computing Systems

What Are We Protecting?

Overall goal: **secure the data** a system manipulates, by enforcing a **security policy**.

- **Confidentiality**: secret data does not leak to non-secret places.
- **Integrity**: high-integrity data is not influenced by low-integrity data.

Secure Computing Systems

What Are We Protecting?

Overall goal: **secure the data** a system manipulates, by enforcing a **security policy**.

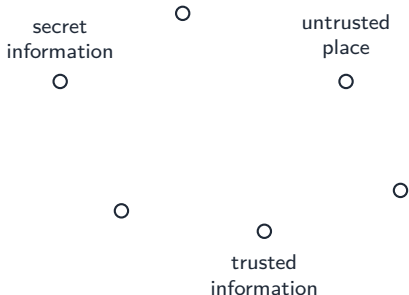
- **Confidentiality**: secret data does not leak to non-secret places.
- **Integrity**: high-integrity data is not influenced by low-integrity data.

Examples: a private key reaching a log file breaks confidentiality; `argv[1]` reaching `system()` breaks integrity.

Information Flow

Can Information Get from One Place to Another?

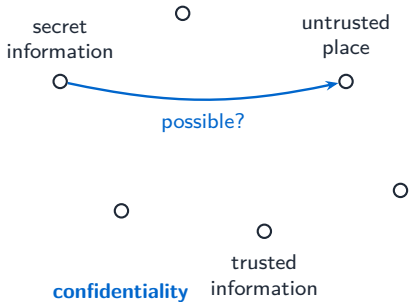
A **place** is anything that holds data: a variable, a memory byte, a register, a file.



Information Flow

Can Information Get from One Place to Another?

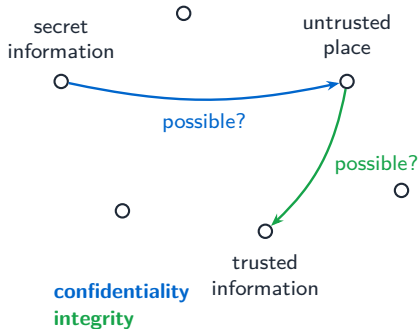
A **place** is anything that holds data: a variable, a memory byte, a register, a file.



Information Flow

Can Information Get from One Place to Another?

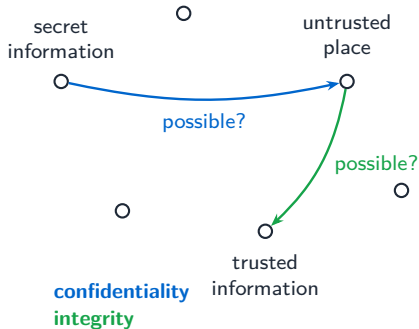
A **place** is anything that holds data: a variable, a memory byte, a register, a file.



Information Flow

Can Information Get from One Place to Another?

A **place** is anything that holds data: a variable, a memory byte, a register, a file.



Example: Confidentiality

The card number must not leak to visible.

```
long cardNb = 1234;
long x = cardNb;
int visible = 0;
if (x > 1000) {
    visible = 1;
}
```

Example: Confidentiality

The card number must not leak to visible.

```
long cardNb = 1234; ← cardNb is the secret
long x = cardNb;
int visible = 0;
if (x > 1000) {
    visible = 1;
}
```

Example: Confidentiality

The card number must not leak to visible.

```
long cardNb = 1234;
long x = cardNb;
int visible = 0;
if (x > 1000) {
    visible = 1;
}
```

← cardNb is the secret

← Secret information propagates to x

Example: Confidentiality

The card number must not leak to visible.

```
long cardNb = 1234;
long x = cardNb;
int visible = 0;
if (x > 1000) {
    visible = 1;
}
```

← cardNb is the secret

← Secret information propagates to x

← Secret information (**partly**) propagates to visible

after the branch, visible says whether the number exceeds 1000.

Example: Integrity

User input must not influence the command executed.

Direct

```
const char *cmd = "ls -l";  
char *x = argv[1];  
cmd = x;  
system(cmd);
```

Example: Integrity

User input must not influence the command executed.

Direct

```
const char *cmd = "ls -l";  
char *x = argv[1];  
cmd = x;  
system(cmd);
```



Low-integrity
data reaches a
high-integrity
variable

Example: Integrity

User input must not influence the command executed.

Direct

```
const char *cmd = "ls -l";  
char *x = argv[1];  
cmd = x;  
system(cmd);
```



Low-integrity
data reaches a
high-integrity
variable

Through a branch

```
const char *cmd = "ls -l";  
char *x = argv[1];  
if (strlen(x) == 5)  
    cmd = "rm -rf *";  
system(cmd);
```

Example: Integrity

User input must not influence the command executed.

Direct

```
const char *cmd = "ls -l";  
char *x = argv[1];  
cmd = x;  
system(cmd);
```



Low-integrity
data reaches a
high-integrity
variable

Through a branch

```
const char *cmd = "ls -l";  
char *x = argv[1];  
if (strlen(x) == 5)  
    cmd = "rm -rf *";  
system(cmd);
```



Untrusted input
must not control
sensitive actions.

Confidentiality vs. Integrity

	Confidentiality	Integrity
Source		
Sink		
Example		

Pradel, Univ. Stuttgart — Information Flow Analysis; Sabelfeld & Myers, IEEE JSAC 2003

Confidentiality vs. Integrity

	Confidentiality	Integrity
Source	secret data	
Sink	public output	
Example	key $\rightarrow$ <code>fprintf(log)</code>	

Confidentiality vs. Integrity

	Confidentiality	Integrity
Source	secret data	untrusted input
Sink	public output	trusted operation
Example	key $\rightarrow$ <code>fprintf(log)</code>	<code>recv</code> $\rightarrow$ system

Confidentiality vs. Integrity

	Confidentiality	Integrity
Source	secret data	untrusted input
Sink	public output	trusted operation
Example	key $\rightarrow$ <code>fprintf(log)</code>	<code>recv</code> $\rightarrow$ <code>system</code>

Confidentiality vs. Integrity

	Confidentiality	Integrity
Source	secret data	untrusted input
Sink	public output	trusted operation
Example	key $\rightarrow$ <code>fprintf(log)</code>	<code>recv</code> $\rightarrow$ <code>system</code>

Tracking Security Labels

How Do We Analyse the Flow?

Tracking Security Labels

How Do We Analyse the Flow?

- Attach to every value a **security label**: meta information recording how secret it is.

Tracking Security Labels

How Do We Analyse the Flow?

- Attach to every value a **security label**: meta information recording how secret it is.
- **Propagate** labels along every program operation.

Tracking Security Labels

How Do We Analyse the Flow?

- Attach to every value a **security label**: meta information recording how secret it is.
- **Propagate** labels along every program operation.
- At a sink, compare the label with what the sink may receive.

Tracking Security Labels

How Do We Analyse the Flow?

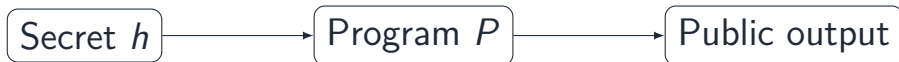
- Attach to every value a **security label**: meta information recording how secret it is.
- **Propagate** labels along every program operation.
- At a sink, compare the label with what the sink may receive.

The same pattern as data-flow analysis: abstract values attached to program state, updated by transfer functions.

Non-Interference

What should confidentiality guarantee?

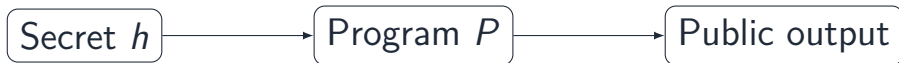
A program may process both **confidential** and **public** information.



Non-Interference

What should confidentiality guarantee?

A program may process both **confidential** and **public** information.

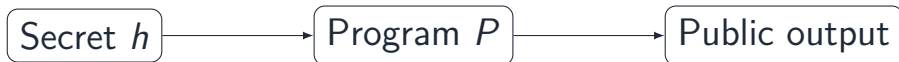


- Keep the **public input fixed**.

Non-Interference

What should confidentiality guarantee?

A program may process both **confidential** and **public** information.

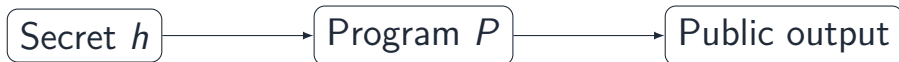


- Keep the **public input fixed**.
- Change the **confidential input**.

Non-Interference

What should confidentiality guarantee?

A program may process both **confidential** and **public** information.

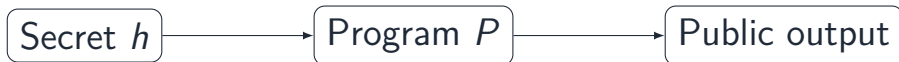


- Keep the **public input fixed**.
- Change the **confidential input**.
- The public observer should see **exactly the same result**.

Non-Interference

What should confidentiality guarantee?

A program may process both **confidential** and **public** information.

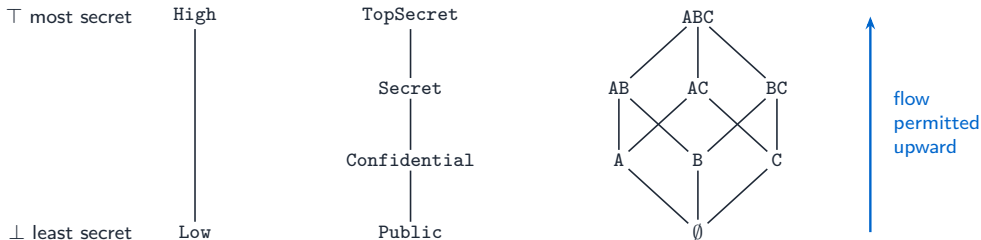


- Keep the **public input fixed**.
- Change the **confidential input**.
- The public observer should see **exactly the same result**.

Lattice of Security Labels

How Do We Represent Levels of Secrecy?

A set of labels arranged in a **lattice**, as in data-flow analysis.



Right: labels are **sets of categories**; the lattice is $2^{\{A,B,C\}}$, so $2^3 = 8$ labels.

Bounded Lattice

Recall. A poset $(S, \sqsubseteq)$ is a **lattice** if every pair a, b has a join $a \sqcup b$ and a meet $a \sqcap b$.

Bounded Lattice

Recall. A poset $(S, \sqsubseteq)$ is a **lattice** if every pair a, b has a join $a \sqcup b$ and a meet $a \sqcap b$.

Bounded additionally requires:

- a **least** element $\perp$ with $\perp \sqsubseteq a$ for every a
- a **greatest** element $\top$ with $a \sqsubseteq \top$ for every a

Bounded Lattice

Recall. A poset $(S, \sqsubseteq)$ is a **lattice** if every pair a, b has a join $a \sqcup b$ and a meet $a \sqcap b$.

Bounded additionally requires:

- a **least** element $\perp$ with $\perp \sqsubseteq a$ for every a
- a **greatest** element $\top$ with $a \sqsubseteq \top$ for every a

$$\boxed{\perp \sqsubseteq a \sqsubseteq \top \quad \text{for every } a \in S}$$

Denning's Lattice Model

A universally bounded lattice is a tuple $(S, \sqsubseteq, \perp, \top, \sqcup, \sqcap)$

S set of security classes

$\sqsubseteq$ partial order: “may flow to”

$\perp$ lower bound

$\top$ upper bound

$\sqcup$ join (least upper bound), $S \times S \rightarrow S$

$\sqcap$ meet (greatest lower bound), $S \times S \rightarrow S$

Denning's Lattice Model

A universally bounded lattice is a tuple $(S, \sqsubseteq, \perp, \top, \sqcup, \sqcap)$

- S set of security classes $\{\emptyset, A, B, C, AB, AC, BC, ABC\}$
- $\sqsubseteq$ partial order: “may flow to”
- $\perp$ lower bound
- $\top$ upper bound
- $\sqcup$ join (least upper bound), $S \times S \rightarrow S$
- $\sqcap$ meet (greatest lower bound), $S \times S \rightarrow S$

Denning's Lattice Model

A universally bounded lattice is a tuple $(S, \sqsubseteq, \perp, \top, \sqcup, \sqcap)$

- S set of security classes $\{\emptyset, A, B, C, AB, AC, BC, ABC\}$
- $\sqsubseteq$ partial order: “may flow to” $\subseteq$
- $\perp$ lower bound
- $\top$ upper bound
- $\sqcup$ join (least upper bound), $S \times S \rightarrow S$
- $\sqcap$ meet (greatest lower bound), $S \times S \rightarrow S$

Denning's Lattice Model

A universally bounded lattice is a tuple $(S, \sqsubseteq, \perp, \top, \sqcup, \sqcap)$

S	set of security classes	$\{\emptyset, A, B, C, AB, AC, BC, ABC\}$
$\sqsubseteq$	partial order: “may flow to”	$\subseteq$
$\perp$	lower bound	$\emptyset$
$\top$	upper bound	ABC
$\sqcup$	join (least upper bound), $S \times S \rightarrow S$	
$\sqcap$	meet (greatest lower bound), $S \times S \rightarrow S$	

Denning's Lattice Model

A universally bounded lattice is a tuple $(S, \sqsubseteq, \perp, \top, \sqcup, \sqcap)$

S	set of security classes	$\{\emptyset, A, B, C, AB, AC, BC, ABC\}$
$\sqsubseteq$	partial order: “may flow to”	$\subseteq$
$\perp$	lower bound	$\emptyset$
$\top$	upper bound	ABC
$\sqcup$	join (least upper bound), $S \times S \rightarrow S$	union: $AB \sqcup A = AB, \emptyset \sqcup AC = AC$
$\sqcap$	meet (greatest lower bound), $S \times S \rightarrow S$	

Denning's Lattice Model

A universally bounded lattice is a tuple $(S, \sqsubseteq, \perp, \top, \sqcup, \sqcap)$

S	set of security classes	$\{\emptyset, A, B, C, AB, AC, BC, ABC\}$
$\sqsubseteq$	partial order: “may flow to”	$\subseteq$
$\perp$	lower bound	$\emptyset$
$\top$	upper bound	ABC
$\sqcup$	join (least upper bound), $S \times S \rightarrow S$	union: $AB \sqcup A = AB$, $\emptyset \sqcup AC = AC$
$\sqcap$	meet (greatest lower bound), $S \times S \rightarrow S$	intersection: $ABC \sqcap C = C$

Denning's Lattice Model

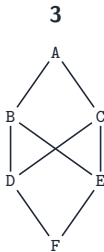
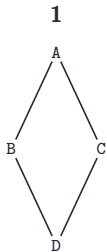
A universally bounded lattice is a tuple $(S, \sqsubseteq, \perp, \top, \sqcup, \sqcap)$

S	set of security classes	$\{\emptyset, A, B, C, AB, AC, BC, ABC\}$
$\sqsubseteq$	partial order: “may flow to”	$\subseteq$
$\perp$	lower bound	$\emptyset$
$\top$	upper bound	ABC
$\sqcup$	join (least upper bound), $S \times S \rightarrow S$	union: $AB \sqcup A = AB, \emptyset \sqcup AC = AC$
$\sqcap$	meet (greatest lower bound), $S \times S \rightarrow S$	intersection: $ABC \sqcap C = C$

$\sqcup$ is the operator that matters: combining two values gives a result **at least as secret as both**.

Quiz: Bounded Lattices

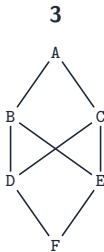
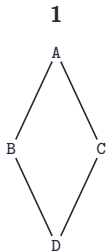
Which of these are bounded lattices?



1. Diamond
2. Chain
3. Crossed
4. Naturals

Quiz: Bounded Lattices

Which of these are bounded lattices?



1. Diamond

YES

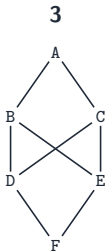
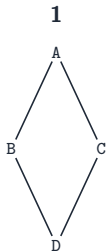
2. Chain

3. Crossed

4. Naturals

Quiz: Bounded Lattices

Which of these are bounded lattices?



1. Diamond

YES

2. Chain

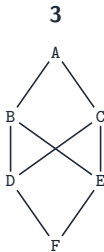
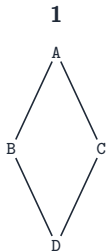
YES

3. Crossed

4. Naturals

Quiz: Bounded Lattices

Which of these are bounded lattices?



1. Diamond **YES**

2. Chain **YES**

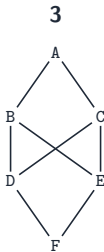
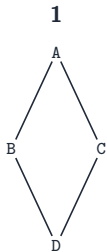
3. Crossed **NO**

Bounded ($\top = A$, $\perp = F$), but
not a lattice: $D \sqcup E$ does not exist.

4. Naturals

Quiz: Bounded Lattices

Which of these are bounded lattices?



1. Diamond **YES**

2. Chain **YES**

3. Crossed **NO**

Bounded ($\top = A$, $\perp = F$), but **not a lattice**: $D \sqcup E$ does not exist.

4. Naturals **NO**

A lattice, but **not bounded**: no $\top$.

Information Flow Policy

What Is Allowed to Flow Where?

```
long cardNb = read_card();  
long x = cardNb;  
int visible = 0;  
if (x > 1000) {  
    visible = 1;  
}  
publish(visible);
```

A policy specifies:

- a lattice of security classes

Information Flow Policy

What Is Allowed to Flow Where?

```
long cardNb = read_card();  
long x = cardNb;  
int visible = 0;  
if (x > 1000) {  
    visible = 1;  
}  
publish(visible);
```

A policy specifies:

- a lattice of security classes
- **sources** of secret information

Information Flow Policy

What Is Allowed to Flow Where?

```
long cardNb = read_card();  
long x = cardNb;  
int visible = 0;  
if (x > 1000) {  
    visible = 1;  
}  
publish(visible);
```

A policy specifies:

- a lattice of security classes
- **sources** of secret information
- **untrusted sinks**

Information Flow Policy

What Is Allowed to Flow Where?

```
long cardNb = read_card();  
long x = cardNb;  
int visible = 0;  
if (x > 1000) {  
    visible = 1;  
}  
publish(visible);
```

Classes: public $\sqsubseteq$ secret.

A policy specifies:

- a lattice of security classes
- **sources** of secret information
- **untrusted sinks**

Goal: **no flow from source to sink.**

Declassification

When “No Flow from High to Low” Is Too Strict

```
char *pw = read_password(); // secret
if (hash(pw) == STORED_HASH) {
    continue_session();
} else {
    puts("incorrect password");
}
```

Declassification

When “No Flow from High to Low” Is Too Strict

```
char *pw = read_password(); // secret
if (hash(pw) == STORED_HASH) {
    continue_session();
} else {
    puts("incorrect password");
}
```

Reveals whether the password
guess was correct.
But: this is **intended**.

Declassification

When “No Flow from High to Low” Is Too Strict

```
char *pw = read_password(); // secret
if (hash(pw) == STORED_HASH) {
    continue_session();
} else {
    puts("incorrect password");
}
```

Reveals whether the password guess was correct.
But: this is **intended**.

Declassification: a mechanism to remove or lower the security class of a value.

Analyzing Information Flows

Given a policy, the analysis checks for **violations**.

Analyzing Information Flows

Given a policy, the analysis checks for **violations**.

- Detect **vulnerable** code: SQL injection, format strings (`printf(buf)`), command injection.

Analyzing Information Flows

Given a policy, the analysis checks for **violations**.

- Detect **vulnerable** code: SQL injection, format strings (`printf(buf)`), command injection.
- Detect **malicious** code: an app sending contacts to a server.

Analyzing Information Flows

Given a policy, the analysis checks for **violations**.

- Detect **vulnerable** code: SQL injection, format strings (`printf(buf)`), command injection.
- Detect **malicious** code: an app sending contacts to a server.
- Check that a program **does not leak**: a key must never be written to the console.

Explicit vs. Implicit Flows

- **Explicit flows**: caused by **data-flow** dependence.
- **Implicit flows**: caused by **control-flow** dependence.

```
long cardNb = 1234;  
long x = cardNb;  
int visible = 0;  
if (x > 1000) {  
    visible = 1;  
}
```

Explicit vs. Implicit Flows

- **Explicit flows**: caused by **data-flow** dependence.
- **Implicit flows**: caused by **control-flow** dependence.

```
long cardNb = 1234;  
long x = cardNb;  
int visible = 0;  
if (x > 1000) {  
    visible = 1;  
}
```

Explicit flow from cardNb to x



Explicit vs. Implicit Flows

- **Explicit flows**: caused by **data-flow** dependence.
- **Implicit flows**: caused by **control-flow** dependence.

```
long cardNb = 1234;  
long x = cardNb;  
int visible = 0;  
if (x > 1000) {  
    visible = 1;  
}
```

Explicit flow from cardNb to x

Implicit flow from x > 1000 to visible

Explicit vs. Implicit Flows

- **Explicit flows**: caused by **data-flow** dependence.  Some analyses consider only these
- **Implicit flows**: caused by **control-flow** dependence.

```
long cardNb = 1234;  
long x = cardNb;  
int visible = 0;  
if (x > 1000) {  
    visible = 1;  
}
```

Explicit flow from cardNb to x



Implicit flow from x > 1000 to visible



Static and Dynamic Analysis

Static information flow analysis

- overapproximates all possible data and control-flow dependences
- result: whether information *may* flow from a secret source to an untrusted sink
- **sound**: never accepts a program that leaks

Static and Dynamic Analysis

Static information flow analysis

- overapproximates all possible data and control-flow dependences
- result: whether information *may* flow from a secret source to an untrusted sink
- **sound**: never accepts a program that leaks

Dynamic information flow analysis

- associates security labels (*taint markings*) with memory locations
- propagates labels at runtime

Static and Dynamic Analysis

Static information flow analysis

- overapproximates all possible data and control-flow dependences
- result: whether information *may* flow from a secret source to an untrusted sink
- **sound**: never accepts a program that leaks

Dynamic information flow analysis

- associates security labels (*taint markings*) with memory locations
- propagates labels at runtime

From here on: **dynamic** analysis, one run at a time.

Taint Sources and Sinks

a **source** is where attacker data enters, a **sink** is where it is dangerous. For confidentiality the source is where *secret* data enters.

Possible sources

- variables
- return values of a function: `getenv`,
`read_pin`
- data from an I/O stream: `read`, `recv`,
`fgets`

Taint Sources and Sinks

a **source** is where attacker data enters, a **sink** is where it is dangerous. For confidentiality the source is where *secret* data enters.

Possible sources

- variables
- return values of a function: `getenv`, `read_pin`
- data from an I/O stream: `read`, `recv`, `fgets`

Possible sinks

- variables
- arguments of a function: `system`, `execve`, `printf`'s format, `send`
- a loop bound or an allocation size
- instructions of a type: indirect `jmp` and `call`, `ret`

Taint Sources and Sinks

a **source** is where attacker data enters, a **sink** is where it is dangerous. For confidentiality the source is where *secret* data enters.

Possible sources

- variables
- return values of a function: `getenv`, `read_pin`
- data from an I/O stream: `read`, `recv`, `fgets`

Possible sinks

- variables
- arguments of a function: `system`, `execve`, `printf`'s format, `send`
- a loop bound or an allocation size
- instructions of a type: indirect `jmp` and `call`, `ret`

Report an **illegal flow** when a taint marking reaches a sink it must not reach.

Taint Propagation: Explicit Flows

Dynamic: as the program runs, for every operation that produces a new value, combine the labels of its inputs:

$$label(result) \leftarrow label(inp_1) \sqcup \dots \sqcup label(inp_k)$$

Taint Propagation: Explicit Flows

Dynamic: as the program runs, for every operation that produces a new value, combine the labels of its inputs:

$$label(result) \leftarrow label(inp_1) \sqcup \dots \sqcup label(inp_k)$$

`z = x + y;` $label(z) = label(x) \sqcup label(y)$

`z = 5;` $label(z) = public$

Taint Propagation: Explicit Flows

Dynamic: as the program runs, for every operation that produces a new value, combine the labels of its inputs:

$$label(result) \leftarrow label(inp_1) \sqcup \dots \sqcup label(inp_k)$$

$z = x + y;$ $label(z) = label(x) \sqcup label(y)$

$z = 5;$ $label(z) = public$

$v = t[i];$ $label(v) = label(t[i]) \sqcup label(i) \text{ ?}$

Taint Propagation: Explicit Flows

Dynamic: as the program runs, for every operation that produces a new value, combine the labels of its inputs:

$$label(result) \leftarrow label(inp_1) \sqcup \dots \sqcup label(inp_k)$$

$z = x + y;$ $label(z) = label(x) \sqcup label(y)$

$z = 5;$ $label(z) = public$

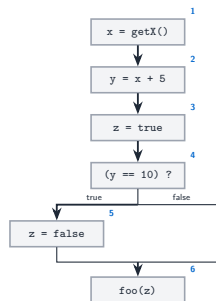
$v = t[i];$ $label(v) = label(t[i]) \sqcup label(i) \text{ ?}$

In C an address is an input too. Whether it counts is a **design choice** of the tool.

Dominators

Which Nodes Must Every Run Pass Through?

d **dominates** n if every path from the entry to n passes through d .



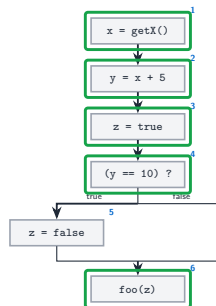
Dominators

Which Nodes Must Every Run Pass Through?

d **dominates** n if every path from the entry to n passes through d .

Every node dominates itself:

$$Dom(6) = \{1, 2, 3, 4, 6\}.$$



Dominators

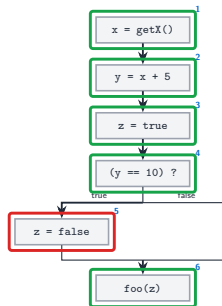
Which Nodes Must Every Run Pass Through?

d **dominates** n if every path from the entry to n passes through d .

Every node dominates itself:

$$Dom(6) = \{1, 2, 3, 4, 6\}.$$

Node 5 does **not** dominate 6: the false edge bypasses it.



Dominators

Which Nodes Must Every Run Pass Through?

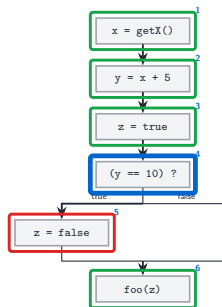
d **dominates** n if every path from the entry to n passes through d .

Every node dominates itself:

$$Dom(6) = \{1, 2, 3, 4, 6\}.$$

Node 5 does **not** dominate 6: the false edge bypasses it.

The **immediate dominator** is the closest strict one: $idom(6) = 4$.



Dominators

Which Nodes Must Every Run Pass Through?

d **dominates** n if every path from the entry to n passes through d .

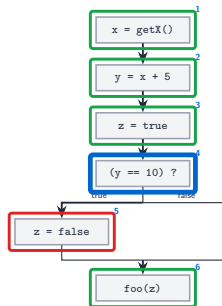
Every node dominates itself:

$$Dom(6) = \{1, 2, 3, 4, 6\}.$$

Node 5 does **not** dominate 6: the false edge bypasses it.

The **immediate dominator** is the closest strict one: $idom(6) = 4$.

In SSA every use must be dominated by its definition.



Dominators as a Data-Flow Analysis

“On *every path from the entry*”: a forward **must** analysis.

Dominators as a Data-Flow Analysis

“On *every path from the entry*”: a forward **must** analysis.

- The facts are the **nodes**: six nodes $\Rightarrow 2^6 = 64$ abstract values.

Dominators as a Data-Flow Analysis

“On *every* path *from the entry*”: a forward **must** analysis.

- The facts are the **nodes**: six nodes $\Rightarrow 2^6 = 64$ abstract values.
- $\text{GEN}[n] = \{n\}$, $\text{KILL}[n] = \emptyset$:

$$\text{IN}[n] = \bigcap_{p \in \text{pred}(n)} \text{OUT}[p] \qquad \text{OUT}[n] = \text{IN}[n] \cup \{n\}$$

Dominators as a Data-Flow Analysis

“On *every path from the entry*”: a forward **must** analysis.

- The facts are the **nodes**: six nodes $\Rightarrow 2^6 = 64$ abstract values.
- $\text{GEN}[n] = \{n\}$, $\text{KILL}[n] = \emptyset$:
$$\text{IN}[n] = \bigcap_{p \in \text{pred}(n)} \text{OUT}[p] \qquad \text{OUT}[n] = \text{IN}[n] \cup \{n\}$$
- $\perp = \text{all nodes}$, $\top = \emptyset$. Sets start *full* (U) and **shrink**.

Dominators as a Data-Flow Analysis

“On every path *from the entry*”: a forward **must** analysis.

- The facts are the **nodes**: six nodes $\Rightarrow 2^6 = 64$ abstract values.
- $\text{GEN}[n] = \{n\}$, $\text{KILL}[n] = \emptyset$:
$$\text{IN}[n] = \bigcap_{p \in \text{pred}(n)} \text{OUT}[p] \qquad \text{OUT}[n] = \text{IN}[n] \cup \{n\}$$
- $\perp = \text{all nodes}$, $\top = \emptyset$. Sets start *full* (U) and **shrink**.
- Boundary: $\text{OUT}[1] = \{1\}$. Result: $\text{Dom}(n) = \text{OUT}[n]$.

Dominators as a Data-Flow Analysis

“On *every path from the entry*”: a forward **must** analysis.

- The facts are the **nodes**: six nodes $\Rightarrow 2^6 = 64$ abstract values.
- $\text{GEN}[n] = \{n\}$, $\text{KILL}[n] = \emptyset$:
$$\text{IN}[n] = \bigcap_{p \in \text{pred}(n)} \text{OUT}[p] \qquad \text{OUT}[n] = \text{IN}[n] \cup \{n\}$$
- $\perp = \text{all nodes}$, $\top = \emptyset$. Sets start *full* (U) and **shrink**.
- Boundary: $\text{OUT}[1] = \{1\}$. Result: $\text{Dom}(n) = \text{OUT}[n]$.

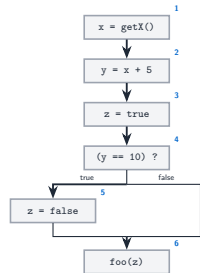
Dominators: Pass 1

Init: all $U = \{1, \dots, 6\}$, except $\text{OUT}[1] = \{1\}$.

Visiting 2...6.

Red marks a value this pass changed.

n	$\text{IN}[n]$	$\text{OUT}[n]$
1	—	$\{1\}$
2	U	U
3	U	U
4	U	U
5	U	U
6	U	U



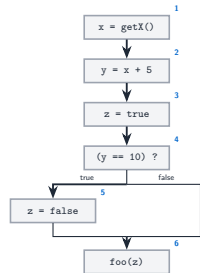
Dominators: Pass 1

Init: all $U = \{1, \dots, 6\}$, except $\text{OUT}[1] = \{1\}$.

Visiting 2...6.

Red marks a value this pass changed.

n	$\text{IN}[n]$	$\text{OUT}[n]$
1	—	$\{1\}$
2	$\{1\}$	$\{1, 2\}$
3	$\{1, 2\}$	$\{1, 2, 3\}$
4	$\{1, 2, 3\}$	$\{1, 2, 3, 4\}$
5	U	U
6	U	U



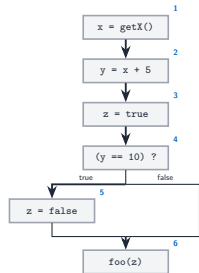
Dominators: Pass 1

Init: all $U = \{1, \dots, 6\}$, except $\text{OUT}[1] = \{1\}$.

Visiting 2...6.

Red marks a value this pass changed.

n	$\text{IN}[n]$	$\text{OUT}[n]$
1	—	$\{1\}$
2	$\{1\}$	$\{1, 2\}$
3	$\{1, 2\}$	$\{1, 2, 3\}$
4	$\{1, 2, 3\}$	$\{1, 2, 3, 4\}$
5	$\{1, 2, 3, 4\}$	$\{1, 2, 3, 4, 5\}$
6	U	U



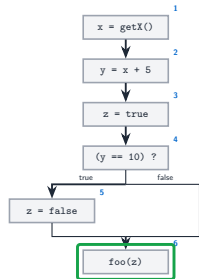
Dominators: Pass 1

Init: all $U = \{1, \dots, 6\}$, except $OUT[1] = \{1\}$.

Visiting 2...6.

Red marks a value this pass changed.

n	$IN[n]$	$OUT[n]$
1	—	$\{1\}$
2	$\{1\}$	$\{1, 2\}$
3	$\{1, 2\}$	$\{1, 2, 3\}$
4	$\{1, 2, 3\}$	$\{1, 2, 3, 4\}$
5	$\{1, 2, 3, 4\}$	$\{1, 2, 3, 4, 5\}$
6	$\{1, 2, 3, 4\}$	$\{1, 2, 3, 4, 6\}$



$$\begin{aligned} IN[6] &= OUT[4] \cap OUT[5] \\ &= \{1, 2, 3, 4\} \cap \{1, 2, 3, 4, 5\} \end{aligned}$$

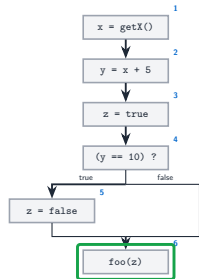
Dominators: Pass 1

Init: all $U = \{1, \dots, 6\}$, except $\text{OUT}[1] = \{1\}$.

Visiting 2...6.

Red marks a value this pass changed.

n	$\text{IN}[n]$	$\text{OUT}[n]$
1	—	$\{1\}$
2	$\{1\}$	$\{1, 2\}$
3	$\{1, 2\}$	$\{1, 2, 3\}$
4	$\{1, 2, 3\}$	$\{1, 2, 3, 4\}$
5	$\{1, 2, 3, 4\}$	$\{1, 2, 3, 4, 5\}$
6	$\{1, 2, 3, 4\}$	$\{1, 2, 3, 4, 6\}$



$$\begin{aligned}\text{IN}[6] &= \text{OUT}[4] \cap \text{OUT}[5] \\ &= \{1, 2, 3, 4\} \cap \{1, 2, 3, 4, 5\}\end{aligned}$$

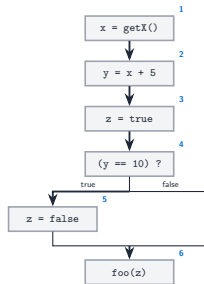
No loop: pass 2 changes **nothing**.

Fixpoint.

Post-Dominators

Where Does a Branch Stop Mattering?

The mirror image: p **post-dominates** n if every path from n to the *exit* passes through p .



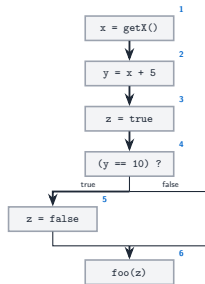
Post-Dominators

Where Does a Branch Stop Mattering?

The mirror image: p **post-dominates** n if every path from n to the *exit* passes through p .

A **backward** must analysis:

$$\text{OUT}[n] = \bigcap_{s \in \text{succ}(n)} \text{IN}[s] \quad \text{IN}[n] = \text{OUT}[n] \cup \{n\}$$



Post-Dominators

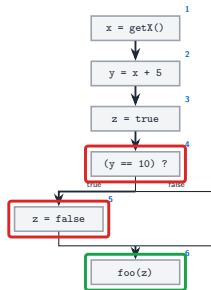
Where Does a Branch Stop Mattering?

The mirror image: p **post-dominates** n if every path from n to the *exit* passes through p .

A **backward** must analysis:

$$\text{OUT}[n] = \bigcap_{s \in \text{succ}(n)} \text{IN}[s] \quad \text{IN}[n] = \text{OUT}[n] \cup \{n\}$$

$PDom(4) = \{4, 6\}$: node 5 is **not** in it.



Post-Dominators

Where Does a Branch Stop Mattering?

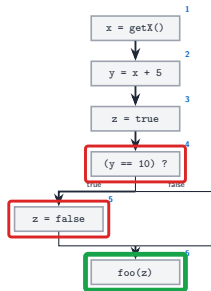
The mirror image: p **post-dominates** n if every path from n to the *exit* passes through p .

A **backward** must analysis:

$$\text{OUT}[n] = \bigcap_{s \in \text{succ}(n)} \text{IN}[s] \quad \text{IN}[n] = \text{OUT}[n] \cup \{n\}$$

$PDom(4) = \{4, 6\}$: node 5 is **not** in it.

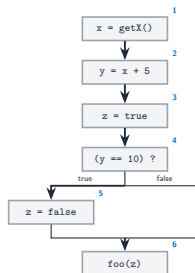
The **immediate post-dominator**, the first node every path from n must reach:
 $ipdom(4) = 6$.



Quiz: Dominators and Post-Dominators

On the graph of Example 2, is it true that:

1. 4 dominates 5?
2. 5 dominates 6?
3. 6 post-dominates 4?



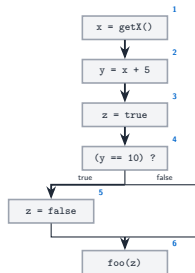
Myers, CS 4120, Cornell — Lecture 25

Quiz: Dominators and Post-Dominators

On the graph of Example 2, is it true that:

1. 4 dominates 5?
2. 5 dominates 6?
3. 6 post-dominates 4?

YES



Myers, CS 4120, Cornell — Lecture 25

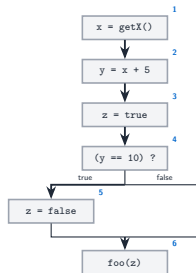
Quiz: Dominators and Post-Dominators

On the graph of Example 2, is it true that:

1. 4 dominates 5?
2. 5 dominates 6?
3. 6 post-dominates 4?

YES

NO



Myers, CS 4120, Cornell — Lecture 25

Quiz: Dominators and Post-Dominators

On the graph of Example 2, is it true that:

1. 4 dominates 5?

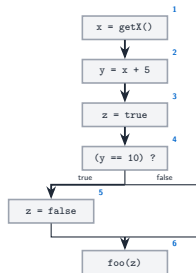
YES

2. 5 dominates 6?

NO

3. 6 post-dominates 4?

YES

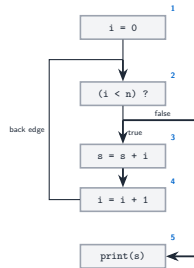


Myers, CS 4120, Cornell — Lecture 25

Quiz: Dominators with a Loop

On this loop, is it true that:

1. 4 dominates 2?
2. 2 dominates 4?
3. 3 dominates 5?
4. 2 post-dominates 4?

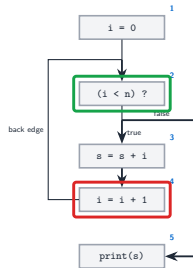


Quiz: Dominators with a Loop

On this loop, is it true that:

1. 4 dominates 2?
2. 2 dominates 4?
3. 3 dominates 5?
4. 2 post-dominates 4?

NO



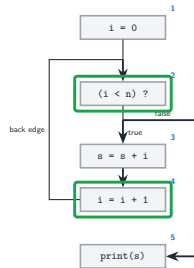
Quiz: Dominators with a Loop

On this loop, is it true that:

1. 4 dominates 2?
2. 2 dominates 4?
3. 3 dominates 5?
4. 2 post-dominates 4?

NO

YES



Quiz: Dominators with a Loop

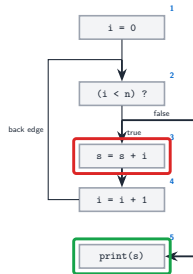
On this loop, is it true that:

1. 4 dominates 2?
2. 2 dominates 4?
3. 3 dominates 5?
4. 2 post-dominates 4?

NO

YES

NO



Quiz: Dominators with a Loop

On this loop, is it true that:

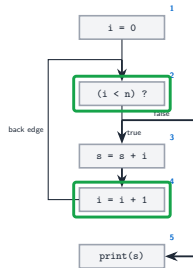
1. 4 dominates 2?
2. 2 dominates 4?
3. 3 dominates 5?
4. 2 post-dominates 4?

NO

YES

NO

YES



Quiz: Dominators with a Loop

On this loop, is it true that:

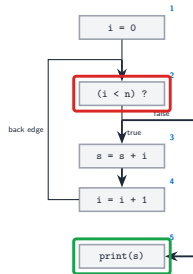
1. 4 dominates 2?
2. 2 dominates 4?
3. 3 dominates 5?
4. 2 post-dominates 4?

NO

YES

NO

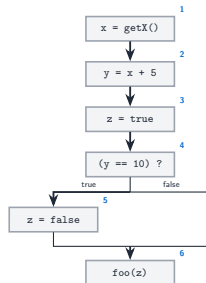
YES



Taint Propagation: Implicit Flows

How Does a Branch Pass On Its Label at Run Time?

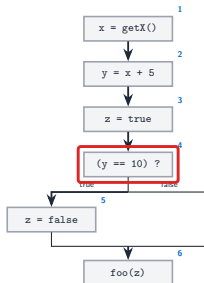
- Node 5 copies nothing from y, yet z reveals y == 10.



Taint Propagation: Implicit Flows

How Does a Branch Pass On Its Label at Run Time?

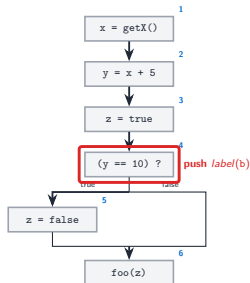
- Node 5 copies nothing from y , yet z reveals $y == 10$.
- b is the **condition**'s value, $b = (y == 10)$; the constant 10 is public:
 $label(b) = label(y) \sqcup public = label(y)$



Taint Propagation: Implicit Flows

How Does a Branch Pass On Its Label at Run Time?

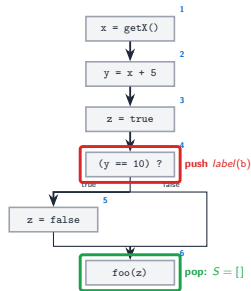
- Node 5 copies nothing from y , yet z reveals $y == 10$.
- b is the **condition**'s value, $b = (y == 10)$; the constant 10 is public:
 $label(b) = label(y) \sqcup public = label(y)$
- Keep a **security stack** S . At the branch, **push** $label(b)$.



Taint Propagation: Implicit Flows

How Does a Branch Pass On Its Label at Run Time?

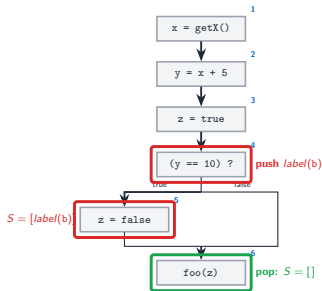
- Node 5 copies nothing from y , yet z reveals $y == 10$.
- b is the **condition**'s value, $b = (y == 10)$; the constant 10 is public:
 $label(b) = label(y) \sqcup public = label(y)$
- Keep a **security stack** S . At the branch, **push** $label(b)$.
- At the **immediate post-dominator** of the branch, **pop** it.



Taint Propagation: Implicit Flows

How Does a Branch Pass On Its Label at Run Time?

- Node 5 copies nothing from y , yet z reveals $y == 10$.
- b is the **condition**'s value, $b = (y == 10)$; the constant 10 is public:
 $label(b) = label(y) \sqcup public = label(y)$
- Keep a **security stack** S . At the branch, **push** $label(b)$.
- At the **immediate post-dominator** of the branch, **pop** it.
- $label(result) = label(inp_1) \sqcup \dots \sqcup label(inp_k) \sqcup \bigsqcup S$
 $\bigsqcup S$: the join of all labels on S .



Example

Dynamic taint analysis: we follow *one run*, with *a* and *c* both true. $label(a) = \text{secret}$, $label(c) = \text{public}$.

	step	now <i>S</i> is	label
if (a) {			
if (c) {			
z = 1;			
}			
w = 2;			
}			

Potential information leak: secret-dependent control flow can make otherwise public values carry secret information.

Example

Dynamic taint analysis: we follow *one run*, with *a* and *c* both true. $label(a) = \text{secret}$, $label(c) = \text{public}$.

	step	now <i>S</i> is	label
if (a) {	push <i>label(a)</i>	[secret]	
if (c) {			
<i>z</i> = 1;			
}			
<i>w</i> = 2;			
}			

Potential information leak: secret-dependent control flow can make otherwise public values carry secret information.

Example

Dynamic taint analysis: we follow *one run*, with *a* and *c* both true. $label(a) = \text{secret}$, $label(c) = \text{public}$.

	step	now <i>S</i> is	label
if (<i>a</i>) {	push <i>label(a)</i>	[secret]	
if (<i>c</i>) {	push <i>label(c)</i>	[secret, public]	
<i>z</i> = 1;			
}			
<i>w</i> = 2;			
}			

Potential information leak: secret-dependent control flow can make otherwise public values carry secret information.

Example

Dynamic taint analysis: we follow *one run*, with *a* and *c* both true. $label(a) = \text{secret}$, $label(c) = \text{public}$.

	step	now <i>S</i> is	label
if (<i>a</i>) {	push $label(a)$	[secret]	
if (<i>c</i>) {	push $label(c)$	[secret , public]	
<i>z</i> = 1;	read <i>S</i>	[secret , public]	$label(z) = \text{public} \sqcup \text{secret} \sqcup \text{public} = \text{secret}$
}			
<i>w</i> = 2;			
}			

Potential information leak: secret-dependent control flow can make otherwise public values carry secret information.

Example

Dynamic taint analysis: we follow *one run*, with *a* and *c* both true. $label(a) = \text{secret}$, $label(c) = \text{public}$.

	step	now <i>S</i> is	label
if (<i>a</i>) {	push <i>label</i> (<i>a</i>)	[secret]	
if (<i>c</i>) {	push <i>label</i> (<i>c</i>)	[secret , public]	
<i>z</i> = 1;	read <i>S</i>	[secret , public]	$label(z) = \text{public} \sqcup \text{secret} \sqcup \text{public} = \text{secret}$
}	pop <i>label</i> (<i>c</i>)	[secret]	
<i>w</i> = 2;			
}			

Potential information leak: secret-dependent control flow can make otherwise public values carry secret information.

Example

Dynamic taint analysis: we follow *one run*, with *a* and *c* both true. $label(a) = \text{secret}$, $label(c) = \text{public}$.

	step	now <i>S</i> is	label
if (<i>a</i>) {	push <i>label</i> (<i>a</i>)	[secret]	
if (<i>c</i>) {	push <i>label</i> (<i>c</i>)	[secret, public]	
<i>z</i> = 1;	read <i>S</i>	[secret, public]	$label(z) = \text{public} \sqcup \text{secret} \sqcup \text{public} = \text{secret}$
}	pop <i>label</i> (<i>c</i>)	[secret]	
<i>w</i> = 2;	read <i>S</i>	[secret]	$label(w) = \text{public} \sqcup \text{secret} = \text{secret}$
}			

Potential information leak: secret-dependent control flow can make otherwise public values carry secret information.

Example

Dynamic taint analysis: we follow *one run*, with *a* and *c* both true. $label(a) = \text{secret}$, $label(c) = \text{public}$.

	step	now <i>S</i> is	label
if (a) {	push <i>label(a)</i>	[secret]	
if (c) {	push <i>label(c)</i>	[secret , public]	
z = 1;	read <i>S</i>	[secret , public]	$label(z) = \text{public} \sqcup \text{secret} \sqcup \text{public} = \text{secret}$
}	pop <i>label(c)</i>	[secret]	
w = 2;	read <i>S</i>	[secret]	$label(w) = \text{public} \sqcup \text{secret} = \text{secret}$
}	pop <i>label(a)</i>	[]	

Potential information leak: secret-dependent control flow can make otherwise public values carry secret information.

Example

Dynamic taint analysis: we follow *one run*, with *a* and *c* both true. $label(a) = \text{secret}$, $label(c) = \text{public}$.

	step	now <i>S</i> is	label
if (a) {	push <i>label(a)</i>	[secret]	
if (c) {	push <i>label(c)</i>	[secret , public]	
z = 1;	read <i>S</i>	[secret , public]	$label(z) = \text{public} \sqcup \text{secret} \sqcup \text{public} = \text{secret}$
}	pop <i>label(c)</i>	[secret]	
w = 2;	read <i>S</i>	[secret]	$label(w) = \text{public} \sqcup \text{secret} = \text{secret}$
}	pop <i>label(a)</i>	[]	

Potential information leak: secret-dependent control flow can make otherwise public values carry secret information.

Example 1: Card Number

Policy: public, **secret**. Source cardNb. Sink publish. **Dynamic**: one run.

```
long cardNb = 1234;
long x = cardNb;
int visible = 0;
if (x > 1000) {
    visible = 1;
}
publish(visible);
```

Example 1: Card Number

Policy: public, **secret**. Source cardNb. Sink publish. **Dynamic**: one run.

```
long cardNb = 1234;  label(cardNb) = secret
long x = cardNb;
int visible = 0;
if (x > 1000) {
    visible = 1;
}
publish(visible);
```

Example 1: Card Number

Policy: public, **secret**. Source cardNb. Sink publish. **Dynamic**: one run.

```
long cardNb = 1234;  label(cardNb) = secret
long x = cardNb;     explicit: label(x) = secret
int visible = 0;
if (x > 1000) {
    visible = 1;
}
publish(visible);
```

Example 1: Card Number

Policy: public, **secret**. Source cardNb. Sink publish. **Dynamic**: one run.

```
long cardNb = 1234;  label(cardNb) = secret
long x = cardNb;     explicit: label(x) = secret
int visible = 0;      label(visible) = public
if (x > 1000) {
    visible = 1;
}
publish(visible);
```

Example 1: Card Number

Policy: public, **secret**. Source cardNb. Sink publish. **Dynamic**: one run.

```
long cardNb = 1234;  label(cardNb) = secret
long x = cardNb;     explicit: label(x) = secret
int visible = 0;     label(visible) = public
if (x > 1000) {      label(b) = secret  $\sqcup$  public = secret; push secret
    visible = 1;
}
publish(visible);
```

b is the value of the condition $x > 1000$: the boolean the branch decides on. Its inputs are x (**secret**) and the constant 1000 (public, like every constant).

Example 1: Card Number

Policy: public, **secret**. Source cardNb. Sink publish. **Dynamic**: one run.

```
long cardNb = 1234;  label(cardNb) = secret
long x = cardNb;     explicit: label(x) = secret
int visible = 0;      label(visible) = public
if (x > 1000) {        label(b) = secret  $\sqcup$  public = secret; push secret
    visible = 1;        label(visible) = public  $\sqcup$  secret = secret
}
publish(visible);
```

b is the value of the condition $x > 1000$: the boolean the branch decides on. Its inputs are x (**secret**) and the constant 1000 (public, like every constant).

Example 1: Card Number

Policy: public, **secret**. Source cardNb. Sink publish. **Dynamic**: one run.

```
long cardNb = 1234;  label(cardNb) = secret
long x = cardNb;     explicit: label(x) = secret
int visible = 0;      label(visible) = public
if (x > 1000) {        label(b) = secret  $\sqcup$  public = secret; push secret
    visible = 1;        label(visible) = public  $\sqcup$  secret = secret
}                       pop
publish(visible);
```

b is the value of the condition $x > 1000$: the boolean the branch decides on. Its inputs are x (**secret**) and the constant 1000 (public, like every constant).

Example 1: Card Number

Policy: public, **secret**. Source cardNb. Sink publish. **Dynamic**: one run.

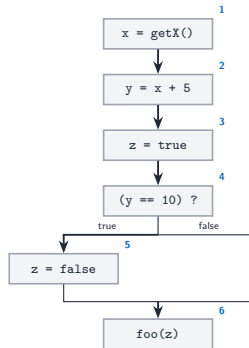
```
long cardNb = 1234;  label(cardNb) = secret
long x = cardNb;     explicit: label(x) = secret
int visible = 0;      label(visible) = public
if (x > 1000) {        label(b) = secret  $\sqcup$  public = secret; push secret
    visible = 1;       label(visible) = public  $\sqcup$  secret = secret
}                      pop
publish(visible);      violation of the policy
```

b is the value of the condition $x > 1000$: the boolean the branch decides on. Its inputs are x (**secret**) and the constant 1000 (public, like every constant).

Quiz: Example 2

Policy: public, **secret**. Source getX(). Sink foo(). **Dynamic** run: getX() returns 5.

```
int x = getX();  
int y = x + 5;  
bool z = true;  
if (y == 10)  
    z = false;  
  
foo(z);
```

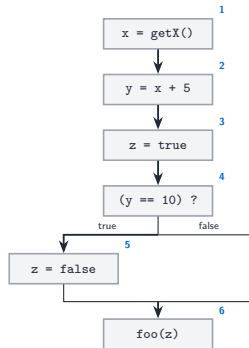


Quiz: Example 2

Policy: public, **secret**. Source getX(). Sink foo(). **Dynamic** run: getX() returns 5.

```
int x = getX(); label(x) = secret
int y = x + 5;
bool z = true;
if (y == 10)
    z = false;

foo(z);
```

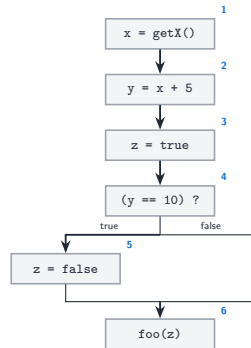


Quiz: Example 2

Policy: public, **secret**. Source `getX()`. Sink `foo()`. **Dynamic** run: `getX()` returns 5.

```
int x = getX();  label(x) = secret
int y = x + 5;   label(y) = secret
bool z = true;
if (y == 10)
    z = false;

foo(z);
```

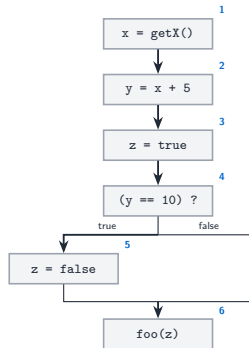


Quiz: Example 2

Policy: public, **secret**. Source `getX()`. Sink `foo()`. **Dynamic** run: `getX()` returns 5.

```
int x = getX();  label(x) = secret
int y = x + 5;   label(y) = secret
bool z = true;   label(z) = public
if (y == 10)
    z = false;

foo(z);
```

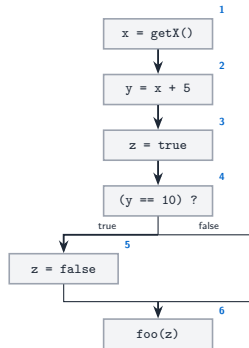


Quiz: Example 2

Policy: public, **secret**. Source getX(). Sink foo(). **Dynamic** run: getX() returns 5.

```
int x = getX();  label(x) = secret
int y = x + 5;   label(y) = secret
bool z = true;   label(z) = public
if (y == 10)     label(b) = secret; push secret
    z = false;

foo(z);
```

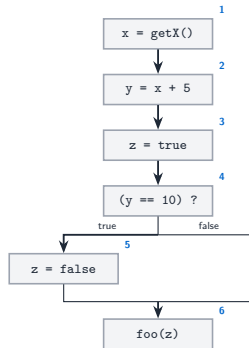


Quiz: Example 2

Policy: public, **secret**. Source getX(). Sink foo(). **Dynamic** run: getX() returns 5.

```
int x = getX();  label(x) = secret
int y = x + 5;   label(y) = secret
bool z = true;   label(z) = public
if (y == 10)     label(b) = secret; push secret
    z = false;   label(z) = secret
```

```
foo(z);
```

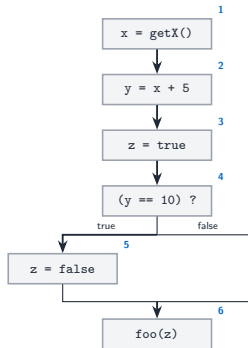


Quiz: Example 2

Policy: public, **secret**. Source getX(). Sink foo(). **Dynamic** run: getX() returns 5.

```
int x = getX();  label(x) = secret
int y = x + 5;   label(y) = secret
bool z = true;   label(z) = public
if (y == 10)     label(b) = secret; push secret
    z = false;   label(z) = secret
                pop secret

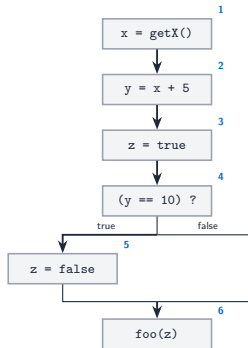
foo(z);
```



Quiz: Example 2

Policy: public, **secret**. Source getX(). Sink foo(). **Dynamic** run: getX() returns 5.

```
int x = getX();  label(x) = secret
int y = x + 5;   label(y) = secret
bool z = true;   label(z) = public
if (y == 10)     label(b) = secret; push secret
    z = false;   label(z) = secret
                pop secret
foo(z);          violation!
```



Hidden Implicit Flows

Example 2 again, one **dynamic** run with `getX()` returning 7.

Hidden Implicit Flows

Example 2 again, one **dynamic** run with `getX()` returning 7.

- `y == 12`: the branch is **not taken**.

Hidden Implicit Flows

Example 2 again, one **dynamic** run with `getX()` returning 7.

- `y == 12`: the branch is **not taken**.
- Nothing is written under the secret.
`label(z)` stays public.

Hidden Implicit Flows

Example 2 again, one **dynamic** run with `getX()` returning 7.

- `y == 12`: the branch is **not taken**.
- Nothing is written under the secret.
`label(z)` stays public.
- `foo(z)`: **no violation reported**.

Hidden Implicit Flows

Example 2 again, one **dynamic** run with `getX()` returning 7.

- `y == 12`: the branch is **not taken**.
- Nothing is written under the secret.
`label(z)` stays public.
- `foo(z)`: **no violation reported**.
- Yet `z == true` tells the attacker `x  $\neq$  5`:
a **false negative**.

Hidden Implicit Flows

Example 2 again, one **dynamic** run with `getX()` returning 7.

- `y == 12`: the branch is **not taken**.
- Nothing is written under the secret. `label(z)` stays public.
- `foo(z)`: **no violation reported**.
- Yet `z == true` tells the attacker `x` $\neq$ 5: a **false negative**.

The smallest case:

```
// label(x) = public
```

```
// label(secret) = secret
```

```
bool x = false;
```

```
if (secret)
```

```
    x = true; ← Copies secret into x
```

But a run with `secret == false` propagates **nothing**

Hidden Implicit Flows: Spurious Definitions

Problem: when `secret` is false, `x = true` never runs, so `x` keeps its public label.

```
bool x = false;  
if (secret)  
    x = true;
```

<i>label(x)</i>	secret true	secret false
without fix	secret	public missed
with <code>x = x</code>		

Hidden Implicit Flows: Spurious Definitions

Problem: when `secret` is false, `x = true` never runs, so `x` keeps its public label.

```
bool x = false;  
if (secret)  
    x = true;  
else  
    x = x; // spurious
```

<i>label(x)</i>	secret true	secret false
without fix	secret	public missed
with <code>x = x</code>		

Fix: make the untaken branch write `x` too, with a no-op `x = x`.

Hidden Implicit Flows: Spurious Definitions

Problem: when `secret` is false, `x = true` never runs, so `x` keeps its public label.

```
bool x = false;  
if (secret)  
    x = true;  
else  
    x = x; // spurious
```

<i>label(x)</i>	secret true	secret false
without fix	secret	public missed
with <code>x = x</code>	secret	

Fix: make the untaken branch write `x` too, with a no-op `x = x`.

Hidden Implicit Flows: Spurious Definitions

Problem: when `secret` is false, `x = true` never runs, so `x` keeps its public label.

```
bool x = false;  
if (secret)  
    x = true;  
else  
    x = x; // spurious
```

<i>label(x)</i>	secret true	secret false
without fix	secret	public missed
with <code>x = x</code>	secret	secret

Fix: make the untaken branch write `x` too, with a no-op `x = x`.

Hidden Implicit Flows: Spurious Definitions

Problem: when `secret` is false, `x = true` never runs, so `x` keeps its public label.

```
bool x = false;
if (secret)
    x = true;
else
    x = x; // spurious
```

<i>label(x)</i>	secret true	secret false
without fix	secret	public missed
with <code>x = x</code>	secret	secret

All executions now propagate the secret label to `x`

Fix: make the untaken branch write `x` too, with a no-op `x = x`.

Hidden Implicit Flows: Spurious Definitions

Problem: when `secret` is false, `x = true` never runs, so `x` keeps its public label.

```
bool x = false;
if (secret)
    x = true;
else
    x = x; // spurious
```

$label(x)$	secret true	secret false
without fix	secret	public missed
with <code>x = x</code>	secret	secret

All executions now propagate the secret label to `x`

Fix: make the untaken branch write `x` too, with a no-op `x = x`.

In general (a **hybrid** step, static before the run): for every conditional, overapproximate what one branch **may define**, and add spurious definitions of it to the other. In C that includes every target of `*p = ...`: a **pointer analysis**.

Implementation in Dytan

Dynamic information flow analysis for **x86 binaries**

Implementation in Dytan

Dynamic information flow analysis for **x86 binaries**

- Taint markings stored as **bit vectors**: one per byte of memory, and per register.

Implementation in Dytan

Dynamic information flow analysis for **x86 binaries**

- Taint markings stored as **bit vectors**: one per byte of memory, and per register.
- Propagation by **instrumentation**: instructions added to the program, via Intel's Pin.

Implementation in Dytan

Dynamic information flow analysis for **x86 binaries**

- Taint markings stored as **bit vectors**: one per byte of memory, and per register.
- Propagation by **instrumentation**: instructions added to the program, via Intel's Pin.
- Immediate post-dominators from the **static control-flow graph**.

Implementation in Dytan

Dynamic information flow analysis for **x86 binaries**

- Taint markings stored as **bit vectors**: one per byte of memory, and per register.
- Propagation by **instrumentation**: instructions added to the program, via Intel's Pin.
- Immediate post-dominators from the **static control-flow graph**.
- **Generic**: sources, sinks and propagation policy are configuration, not code.

Implementation in Dytan

Dynamic information flow analysis for **x86 binaries**

- Taint markings stored as **bit vectors**: one per byte of memory, and per register.
- Propagation by **instrumentation**: instructions added to the program, via Intel's Pin.
- Immediate post-dominators from the **static control-flow graph**.
- **Generic**: sources, sinks and propagation policy are configuration, not code.

Language-independent: it runs compiled C, C++, or anything else that becomes x86.

Taint Tracking for C/C++: DataFlowSanitizer

Dynamic, built into Clang: `clang -fsanitize=dataflow`

```
#include <sanitizer/dfsan_interface.h>
int i = 100, j = 200, k = 300;
dfsan_set_label(1, &i, sizeof i);
dfsan_set_label(2, &j, sizeof j);
dfsan_set_label(4, &k, sizeof k);
dfsan_label ij = dfsan_get_label(i+j);
```

Taint Tracking for C/C++: DataFlowSanitizer

Dynamic, built into Clang: `clang -fsanitize=dataflow`

```
#include <sanitizer/dfsan_interface.h>
int i = 100, j = 200, k = 300;
dfsan_set_label(1, &i, sizeof i);
dfsan_set_label(2, &j, sizeof j);
dfsan_set_label(4, &k, sizeof k);
dfsan_label ij = dfsan_get_label(i+j);
```

Result: `ij == 3`: the labels of `i` and `j`, not `k`.

Taint Tracking for C/C++: DataFlowSanitizer

Dynamic, built into Clang: `clang -fsanitize=dataflow`

```
#include <sanitizer/dfsan_interface.h>
int i = 100, j = 200, k = 300;
dfsan_set_label(1, &i, sizeof i);
dfsan_set_label(2, &j, sizeof j);
dfsan_set_label(4, &k, sizeof k);
dfsan_label ij = dfsan_get_label(i+j);
```

Result: `ij == 3`: the labels of `i` and `j`, not `k`.

How it works

1. `dfsan_set_label` stores a label in **shadow memory**, beside each byte of `i`.
2. The compiler adds code next to every operation: for `i + j`, label of result = $label(i) \sqcup label(j)$.
3. `dfsan_get_label` reads the label back.

Taint Tracking for C/C++: DataFlowSanitizer

Dynamic, built into Clang: `clang -fsanitize=dataflow`

```
#include <sanitizer/dfsan_interface.h>
int i = 100, j = 200, k = 300;
dfsan_set_label(1, &i, sizeof i);
dfsan_set_label(2, &j, sizeof j);
dfsan_set_label(4, &k, sizeof k);
dfsan_label ij = dfsan_get_label(i+j);
```

Result: `ij == 3`: the labels of `i` and `j`, not `k`.

How it works

1. `dfsan_set_label` stores a label in **shadow memory**, beside each byte of `i`.
2. The compiler adds code next to every operation: for `i + j`, label of result = $label(i) \sqcup label(j)$.
3. `dfsan_get_label` reads the label back.

Our lattice $2^{\{A,B,C\}}$, with 8 categories instead of 3.

Clang DataFlowSanitizer docs

Static Taint Analysis for C/C++

Static: the Clang Static Analyzer, no execution needed

```
int main(void) {  
    char cmd[256];  
    scanf("%255s", cmd);  
    system(cmd);  
}
```

Clang Static Analyzer checkers

Static Taint Analysis for C/C++

Static: the Clang Static Analyzer, no execution needed

```
int main(void) {  
    char cmd[256];  
    scanf("%255s", cmd);  
    system(cmd);  
}
```

```
clang -analyze \  
-Xclang -analyzer-checker=\  
optin.taint.GenericTaint t.c
```

Clang Static Analyzer checkers

Static Taint Analysis for C/C++

Static: the Clang Static Analyzer, no execution needed

```
int main(void) {  
    char cmd[256];  
    scanf("%255s", cmd);  
    system(cmd);  
}
```

```
clang -analyze \  
-Xclang -analyzer-checker=\  
optin.taint.GenericTaint t.c
```

warning: Untrusted data is passed to a system call
(CERT/STR02-C. Sanitize data passed to complex
subsystems)

Clang Static Analyzer checkers

Static Taint Analysis for C/C++

Static: the Clang Static Analyzer, no execution needed

```
int main(void) {  
    char cmd[256];  
    scanf("%255s", cmd);  
    system(cmd);  
}
```

```
clang -analyze \  
-Xclang -analyzer-checker=\  
optin.taint.GenericTaint t.c
```

warning: Untrusted data is passed to a system call
(CERT/STR02-C. Sanitize data passed to complex
subsystems)

Source scanf, sink system: an **integrity** policy.

Clang Static Analyzer checkers

Other Tools

The same lattice, sources, sinks and propagation rules.

Target	Dynamic	Static
x86 binaries		
C / C++		
Java / Android		
Python		
Perl		
JavaScript		

Clause, Li & Orso, ISSTA 2007; Kemerlis et al., VEE 2012; Enck et al., ACM TOCS 2014; Arzt et al., PLDI 2014; Myers, POPL 1999

Other Tools

The same lattice, sources, sinks and propagation rules.

Target	Dynamic	Static
x86 binaries	Dytan, libdft, TaintCheck	
C / C++	DataFlowSanitizer	Clang Static Analyzer, CodeQL
Java / Android		
Python		
Perl		
JavaScript		

Clause, Li & Orso, ISSTA 2007; Kemerlis et al., VEE 2012; Enck et al., ACM TOCS 2014; Arzt et al., PLDI 2014; Myers, POPL 1999

Other Tools

The same lattice, sources, sinks and propagation rules.

Target	Dynamic	Static
x86 binaries	Dytan, libdft, TaintCheck	
C / C++	DataFlowSanitizer	Clang Static Analyzer, CodeQL
Java / Android	TaintDroid	FlowDroid; Jif (type system)
Python		
Perl		
JavaScript		

Clause, Li & Orso, ISSTA 2007; Kemerlis et al., VEE 2012; Enck et al., ACM TOCS 2014; Arzt et al., PLDI 2014; Myers, POPL 1999

Other Tools

The same lattice, sources, sinks and propagation rules.

Target	Dynamic	Static
x86 binaries	Dytan, libdft, TaintCheck	
C / C++	DataFlowSanitizer	Clang Static Analyzer, CodeQL
Java / Android	TaintDroid	FlowDroid; Jif (type system)
Python		Pysa, CodeQL
Perl	taint mode, perl -T	
JavaScript		CodeQL

Clause, Li & Orso, ISSTA 2007; Kemerlis et al., VEE 2012; Enck et al., ACM TOCS 2014; Arzt et al., PLDI 2014; Myers, POPL 1999

Other Tools

The same lattice, sources, sinks and propagation rules.

Target	Dynamic	Static
x86 binaries	Dytan, libdft, TaintCheck	
C / C++	DataFlowSanitizer	Clang Static Analyzer, CodeQL
Java / Android	TaintDroid	FlowDroid; Jif (type system)
Python		Pysa, CodeQL
Perl	taint mode, perl -T	
JavaScript		CodeQL

What changes per language: **what a place is**, and which operations exist.

Clause, Li & Orso, ISSTA 2007; Kemerlis et al., VEE 2012; Enck et al., ACM TOCS 2014; Arzt et al., PLDI 2014; Myers, POPL 1999